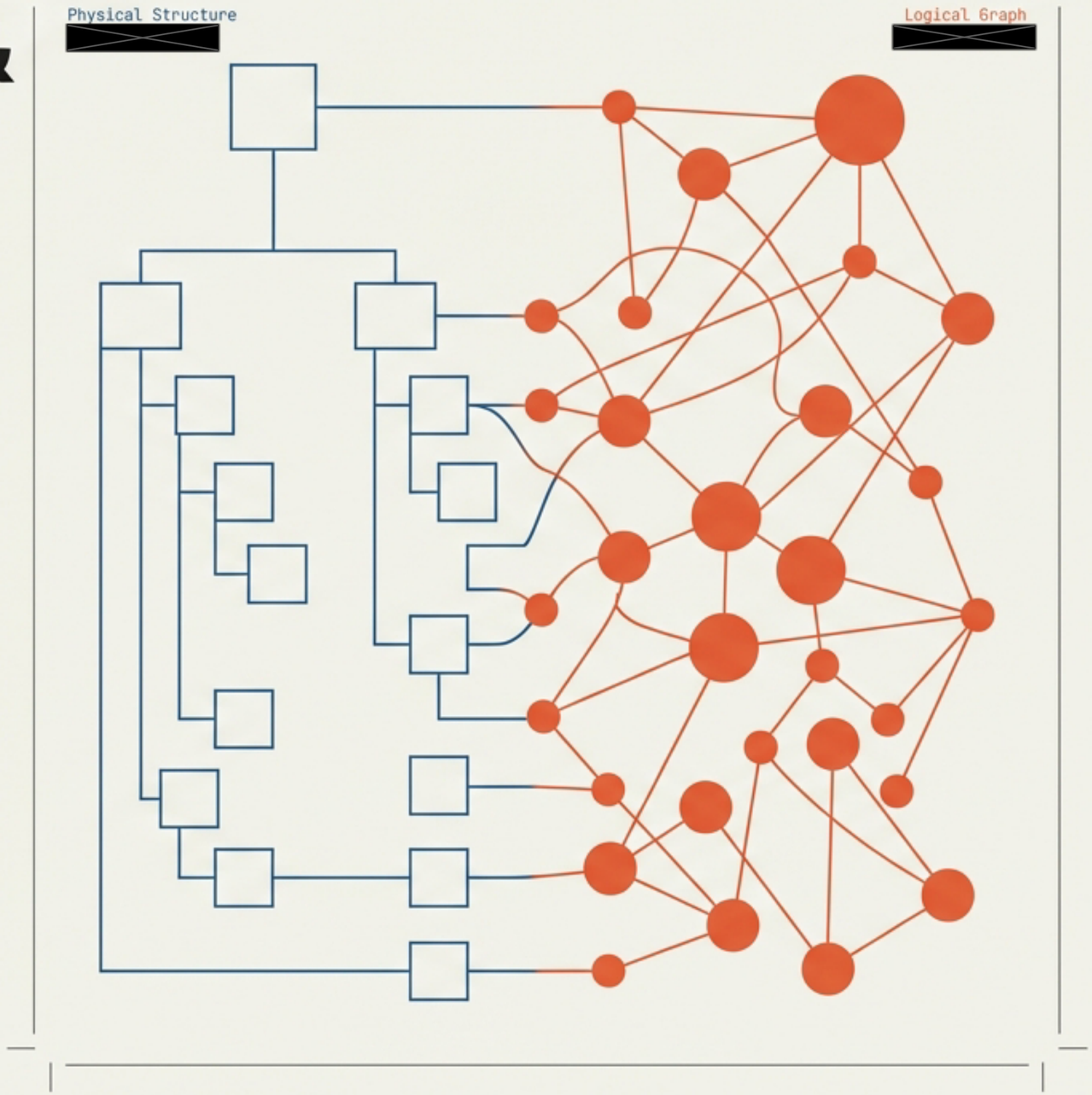
Hierarchical Folders & Graph Integrity

[redacted]

Architectural Proposal v0.2.26 | Enhancing Issue Tracking for Scalability and AI



[redacted]

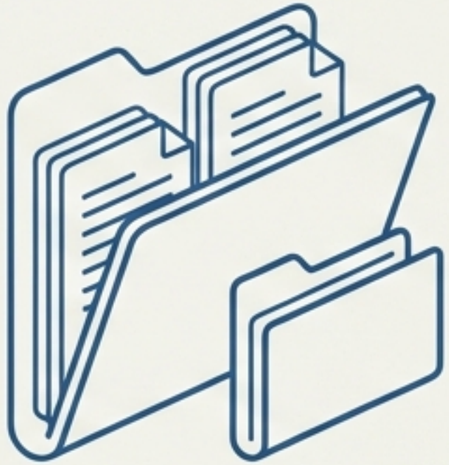
DATE: 2026-02-01
STATUS: Proposal for Review (Draft v0.1)
RELATED: ARCH-001 (MGraph-DB Integration)

The Twin Pillars of Data Integrity

This proposal introduces two interconnected enhancements that transform the issue tracking system from a flat list into a navigable, intelligent ecosystem.

Pillar 1: Hierarchical Folders

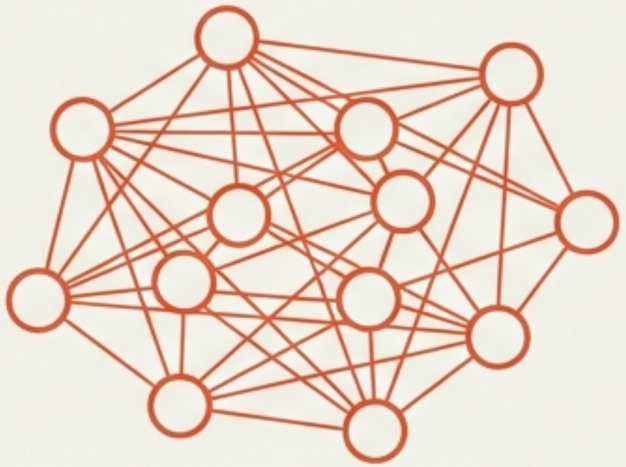
The Physical Pillar



- **Concept:** Issues organized in nested folders mirroring logical relationships.
- **Core Value:** Natural containment. Each folder acts as a self-contained 'mini-database'.

Pillar 2: No-Orphan Principle

The Logical Pillar

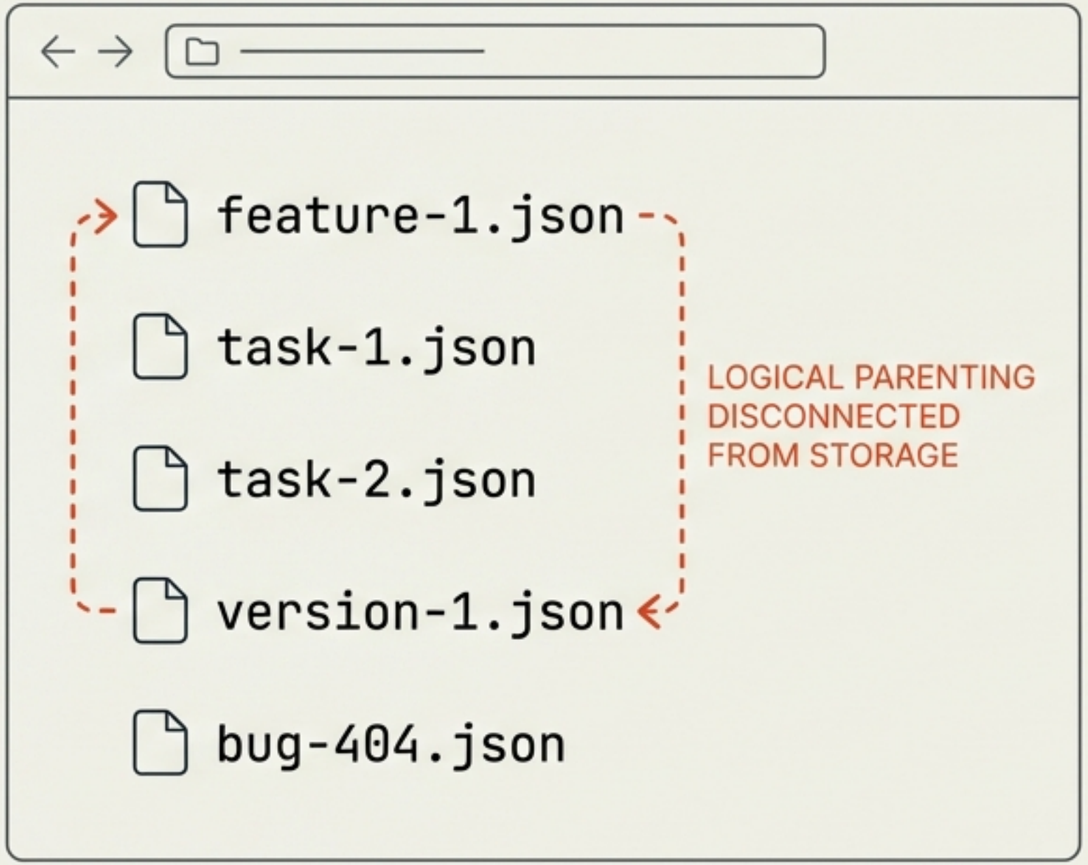


- **Concept:** A mandate that every issue must connect to at least one other.
- **Core Value:** Complete reachability. Ensures a fully connected graph traversable by AI agents.

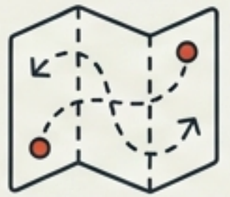
THE COMBINED OUTCOME: A SYSTEM WHERE DATA IS PHYSICALLY PORTABLE AND LOGICALLY UNBREAKABLE.

The Friction of Flat Structure

The Visual Problem



The Consequences



Poor Locality

Related files are scattered, making manual navigation difficult.

Rigid Scoping

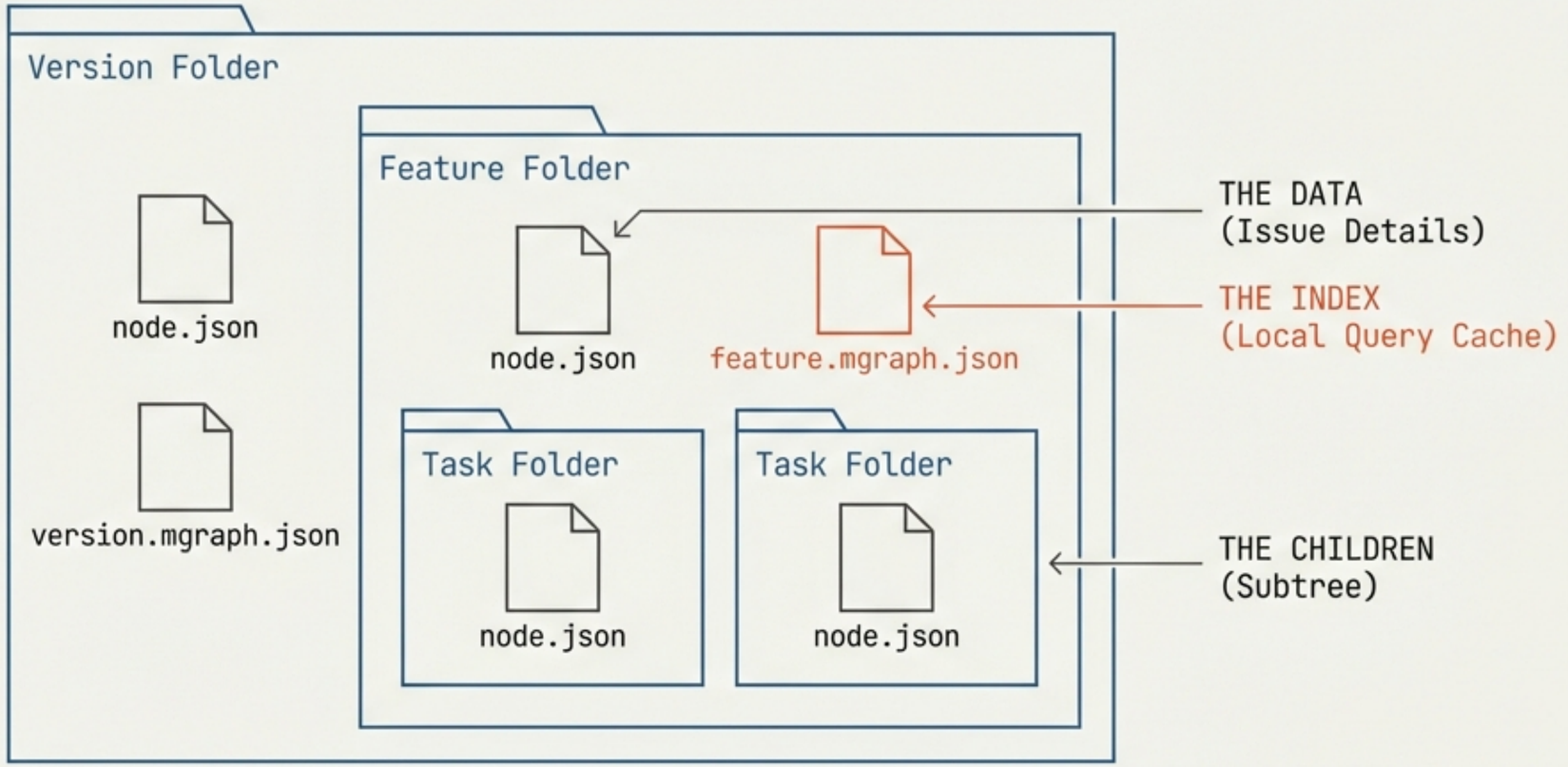
Moving a feature requires finding and moving loosely associated files individually.

AI Blindness

Agents lack a ‘bounded context’ and must scan the entire database to understand a specific feature.

Pillar 1: The Hierarchical Shift

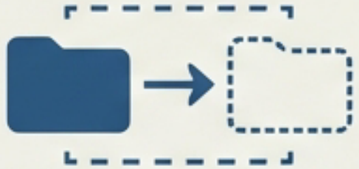
The ‘Mini-Database’ Mental Model



KEY INSIGHT: A folder containing an issue becomes a SELF-CONTAINED UNIT. It holds its own data, its index, and all relationships within its subtree.

Benefits of Structural Locality

Portability



[redacted]

Move a feature folder = move all its bugs, tasks, and history instantly.

Git-Friendliness

[redacted]

Branching a feature folder in Git automatically branches the entire context of that feature.

Natural Scoping

[redacted]

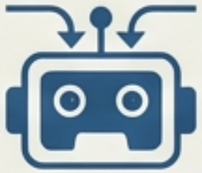
“Feature-1 /” contains strictly everything regarding Feature-1.

Optimised Indexing

[redacted]

Indexes are smaller and faster because they only cover their specific subtree, not the global database.

AI Context



[redacted]

An agent can be pointed to a specific directory to gain “focused context” without noise.

Indexing Strategy & Granularity

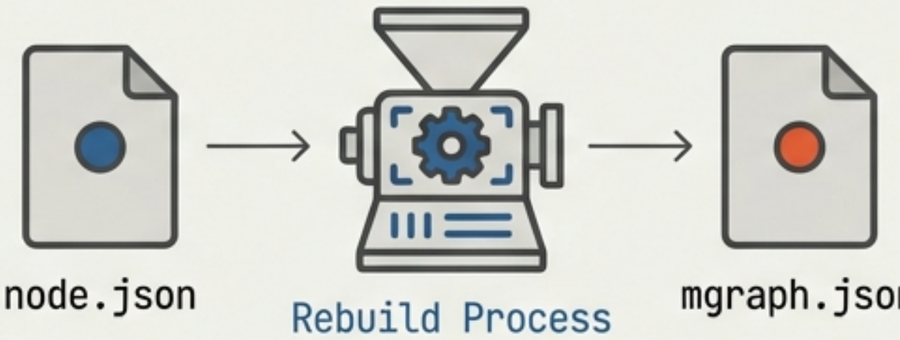
Rule: Create {type}.mgraph.json when a folder has 5+ descendants or requires independent querying.

Type	Create Index?	Rationale
Versions & Features	YES	Major organizational units.
User Stories	MAYBE	If they contain many tasks.
Tasks & Bugs	RARELY	Usually leaf nodes.

Technical Insight

INDEX != DATABASE

Indexes are purely for performance optimization. If deleted, they can be rebuilt from the source 'node.json' files. They are not the source of truth.



Pillar 2: The No-Orphan Principle

“There should never be an issue that does not have a link to something else.”

DEFINITION: ORPHAN

A node with zero incoming and zero outgoing links. Invisible to traversal.

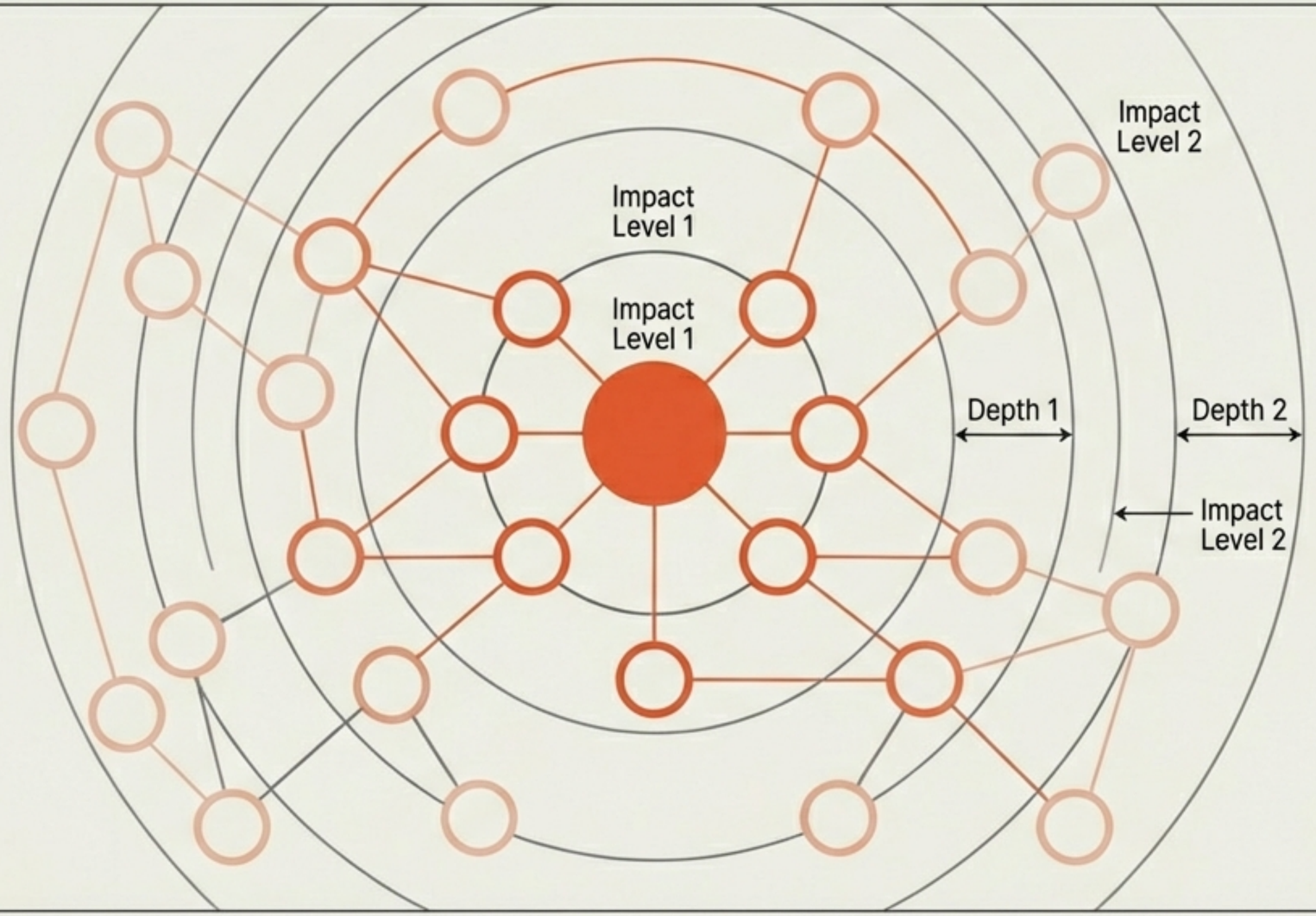
DEFINITION: CONNECTED GRAPH

A system where every node is reachable via traversal from a ‘Root’.

The Uber Global Parent concept ensures the entire system is reachable from a single root for complete AI visibility.

Visualising Impact: The 'Blast Radius'

Leveraging graph integrity for decision intelligence.



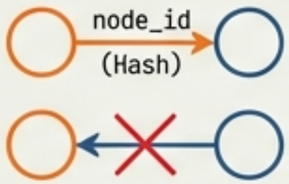
Enabled Metrics

1. **Graph Health**
   Percentage of reachable nodes.
2. **Orphan Lists**
   Automated detection of disconnected work.
3. **Root Identification**
   Trace any task back to its high-level strategic goal.

Link Logic & Taxonomy

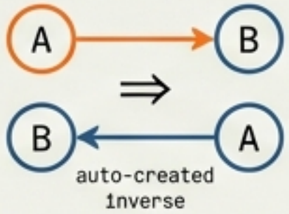
The Core Rules

1. **Identity**
   Links reference "node_id" (Hash), NOT unstable labels.



2. **Directionality**
   All links are strictly directed (A -> B).

3. **Bidirectionality**
   System auto-creates the inverse (A -> B implies B <- A).



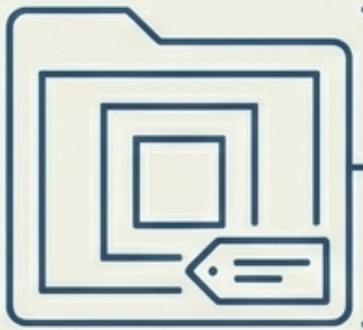
Taxonomy Table

Verb (Forward)	Inverse (Backward)	Usage
has-task	task-of	Containment
blocks	blocked-by	Dependency
depends-on	dependency-of	Dependency
implements	implemented-by	Traceability

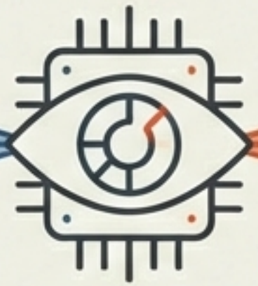
BANNED: Symmetric links (e.g., "relates-to") are forbidden. They obscure dependency flow.

Synthesis: Enabling AI & Agent Interaction

BOUNDED CONTEXT

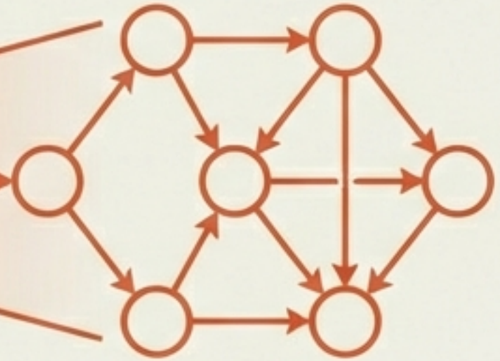


Hierarchy tells the agent WHERE to look.
It loads only relevant files.



Agent

SEMANTIC CONTEXT



Graph links tell the agent WHAT IT MEANS.
It understands 'blocks' vs 'implements'.

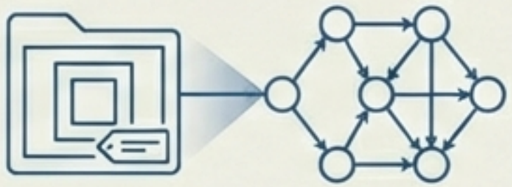
Result: Agents can ‘understand’ a feature’s scope and status without ingesting the entire project database.

Architectural Design Decisions

Hybrid Approach

Support both Flat and Hierarchical structures simultaneously during transition.

Allows the system to manage its own evolution.

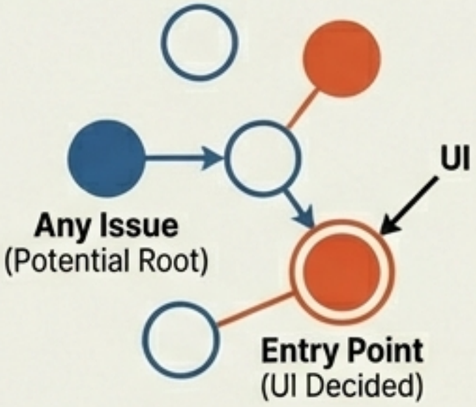


Root Nodes

No special ‘Project’ type required.

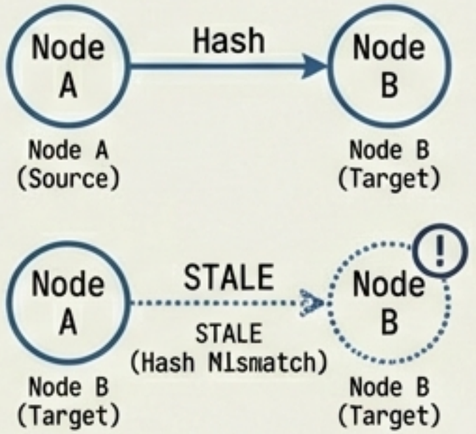
Any issue can serve as a root.

The UI decides the entry point.



Change Tracking

Links track the hash of the target issue. If a target changes, the link becomes “stale”, triggering notifications.



Label Collisions

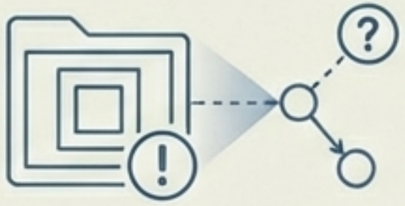
Resolution strategies will be detailed in an upcoming voice memo.

Implementation Roadmap

Phase 1: No-Orphan Detection

Quick Win

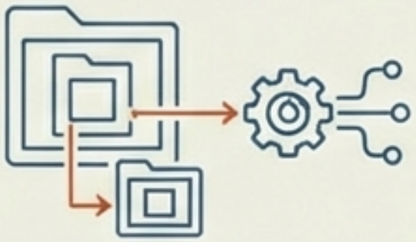
Add detection to flat structure.
API: /nodes/api/orphans.
UI Warning badges.



Phase 3: Hierarchical Folders

Major Change

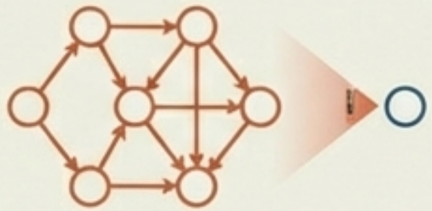
Migration design.
Updating MGraph index builders and file watchers.



Phase 2: Graph Health Dashboard

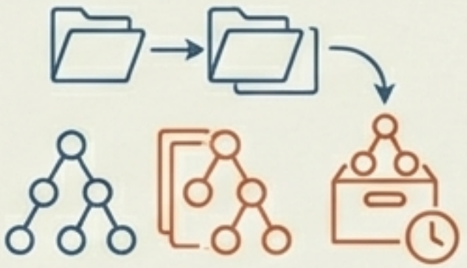
Visualization

Visualizing connected components and orphan orphan resolution workflows.



Phase 4: Subtree Operations

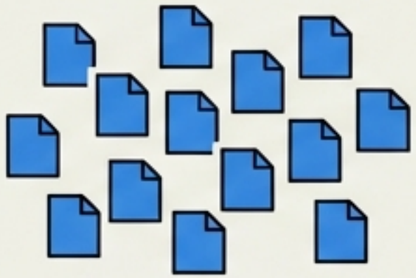
Migration design.
Updating MGraph index builders and file watchers for nested paths.



Migration Strategy

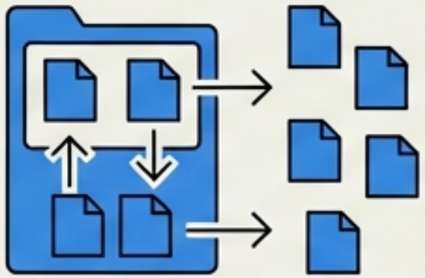
From Flat to **Hierarchical** with Backward Compatibility

State A: Flat



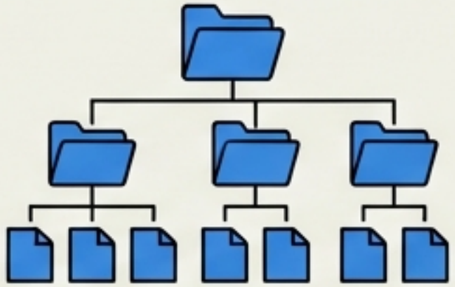
Scattered files

Transition: Hybrid



Some files nested, some flat

State B: Hierarchical



Fully nested tree

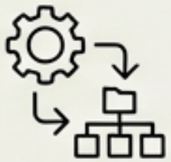
Key Technical Updates

File Watchers

Must be updated to respect recursive directory paths.

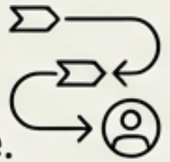
MGraph Builders

Must be updated to scan sub-directories.



Strategy

Gradual shift. Users adopt hierarchy feature-by-feature.



API & Integration Specs

Graph Integrity

```
GET /nodes/api/orphans     // List disconnected nodes
GET /nodes/api/roots      // List intentional root nodes
GET /nodes/api/graph/stats // Health metrics
```

Visualization

```
GET /nodes/api/{type}/{label}/blast-radius?depth=N
// Returns transitive impact graph for visualization
```

AI Assistance

```
GET /nodes/api/{type}/{label}/suggest-links
// AI analyzes content to suggest connections
```

AI Assistance

```
POST /nodes/api/{type}/{label}/suggest-links
// AI analyzes content to suggest missing connections
```

Summary & Next Steps

Current Status: Proposal for Review

Next Steps

- [ ] Review Brief-001 and Brief-002 for compatibility.
- [ ] Prototype orphan detection (Phase 1).
- [ ] Record Voice Memo #3 (Label Conflict Resolution).

FINAL DEFINITION:

“We are building a system that is PHYSICALLY NESTED for containment and LOGICALLY CONNECTED for integrity.”